Boggle Algorithm Benchmark: Complete Explanation

1. What this project does

This project solves a 4x4 Boggle-style word board and compares six different search strategies:

1. Brute-force DFS
2. Prefix-set DFS
3. Trie DFS
4. Dictionary-driven DFS
5. Frequency-filtered Trie DFS
6. Trie DFS with subtree pruning

The program is written in Python. Streamlit provides the user interface, so no separate HTML, CSS, JavaScript, or web server is needed.

The user can:

- edit the 16 board cells;
- choose a minimum word length;
- use the bundled dictionary or upload a text dictionary;
- run all six algorithms;
- compare their timings and search statistics;
- inspect a word's path on the board;
- read a short explanation and complexity note for every algorithm.

The main source file is `server.py`. The name is historical: it is no longer an HTTP server. It is the Streamlit application and the complete benchmark implementation.

---

2. How to run the project

From the project folder:

```
python -m pip install -r requirements.txt
streamlit run server.py
```

Then open the local URL printed by Streamlit, normally:

```
http://localhost:8501
```

The bundled dictionary is:

```
data/default_dictionary.txt
```

Each dictionary line is treated as one possible word.

---

3. High-level program flow

The complete flow is:

```
Start Streamlit
      |
      v
Build the 4x4 input grid
      |
      v
User edits cells and clicks "Run benchmark"
      |
      v
Join the cells into four rows
      |
      v
Validate and normalize the board
      |
      v
Load the default or uploaded dictionary
      |
      v
Normalize dictionary words
      |
      v
Run all six algorithms
      |
      v
Store results in Streamlit session state
      |
      v
Display table, charts, word paths, and explanations
```

The search algorithms all receive the same logical inputs:

- the board letters as a flat list of 16 characters;
- a set of eligible dictionary words;
- a map of neighboring board cells.

This is important for a fair comparison. The algorithms differ in how they search, not in the board or dictionary they receive.

---

4. The main file and imports

`server.py` imports:

```
from dataclasses import asdict, dataclass
from pathlib import Path
from time import perf_counter
from typing import Callable

import pandas as pd
import streamlit as st
```

`dataclass`

`dataclass` is used for `Result`. It creates a simple object for holding the metrics returned by one algorithm.

`asdict`

`asdict(result)` converts a `Result` object into a dictionary. Streamlit can display a list of dictionaries as a table.

`Path`

`Path` creates a safe, platform-independent path to the bundled dictionary.

`perf_counter`

`perf_counter()` measures elapsed time. It is appropriate for benchmarking because it is designed for measuring short durations.

`Callable`

This is a type hint. It states that `run` receives a search function.

`pandas`

Pandas is used only to prepare the data for Streamlit's charts.

`streamlit`

Streamlit creates the interface and reruns the Python script when the user interacts with it.

---

5. Constants and algorithm descriptions

```
SIZE = 4
ROOT = Path(__file__).parent
DEFAULT_FILE = ROOT / "data" / "default_dictionary.txt"
```

- `SIZE` is 4 because the board is fixed at 4x4.
- `ROOT` is the folder containing `server.py`.
- `DEFAULT_FILE` points to the bundled dictionary.

`ALGORITHM_INFO` is a dictionary used by the interface. It maps each algorithm name to:

1. a plain-English explanation;
2. a complexity description.

It does not control the algorithms. It only supplies educational text for the expander in the interface.

---

6. The `Result` data class

```
@dataclass
class Result:
    name: str
    words: dict[str, list[tuple[int, int]]]
    prep: float
    search: float
    states: int
    checks: int
    pruned: int
    filtered: int = 0
```

One `Result` represents one algorithm's output.

Fields

`name`

The display name, such as `Trie DFS`.

`words`

A dictionary of discovered words and one valid path for each word.

Example:

```
{
    "CAT": [(0, 0), (0, 1), (0, 2)]
}
```

The coordinates are zero-based internally. The interface adds one when displaying them to the user.

`prep`

Time spent preparing data, such as filtering dictionary words or building a trie.

`search`

Time spent actually traversing the board.

`states`

How many DFS states were visited. A state represents a recursive search position, including the current cell, path, and search data.

`checks`

How many neighboring-cell possibilities were examined.

`pruned`

How many possible branches were rejected before continuing.

Examples of pruning:

- a cell was already visited;

- a character did not match the next word character;
- a partial string was not a dictionary prefix;
- a trie branch did not exist.

`filtered`

How many candidate words were removed before searching. This is mainly useful for the frequency-filtered algorithm.

---

7. Board representation

The user sees a 4x4 grid, but the algorithms use a flat list:

```
C A T S
R A T E
L I O N
P E N D
```

becomes:

```
["C", "A", "T", "S", "R", "A", "T", "E",
 "L", "I", "O", "N", "P", "E", "N", "D"]
```

The index of a cell is:

```
index = row * 4 + column
```

For example:

- index 0 is row 0, column 0;
- index 3 is row 0, column 3;
- index 4 is row 1, column 0;
- index 15 is row 3, column 3.

This representation makes visited-cell tracking with a bit mask possible.

---

8. Dictionary normalization: `words_from`

```
def words_from(lines: list[str]) -> tuple[set[str], int]:
```

This function receives dictionary lines and returns:

```
(valid_words, rejected_count)
```

For each line it:

1. removes surrounding whitespace;
2. checks that it is not empty;
3. checks that it contains only ASCII letters;
4. converts it to uppercase;
5. adds it to a set.

A set is used because:

- duplicate words should not be searched repeatedly;
- membership checks such as `word in candidates` are fast on average;
- words are automatically unique.

Invalid lines increase `rejected`. Examples include:

- blank lines;
- numbers;
- punctuation;
- accented or non-ASCII characters.

The use of `set` means duplicate dictionary lines are not counted as separate words. The reported valid dictionary count is the number of unique valid words.

---

9. Board validation: `board_from`

```
def board_from(text: str) -> list[list[str]]:
```

This function validates the board entered in the interface.

It checks:

1. there are exactly four rows;
2. every row has exactly four characters;
3. every character is alphabetic;
4. all characters are converted to uppercase.

For valid input it returns a matrix:

```
[
    ["C", "A", "T", "S"],
    ["R", "A", "T", "E"],
    ["L", "I", "O", "N"],
    ["P", "E", "N", "D"],
]
```

If validation fails, it raises `ValueError`. The Streamlit interface catches that error and displays it to the user.

The cells already limit each input to one character, but this validation is still necessary because it is the final protection before the algorithms run.

---

10. Board adjacency: `neighbors`

```
def neighbors() -> dict[int, list[int]]:
```

Boggle allows movement to the eight surrounding cells:

```
top-left   top   top-right
left       cell  right
bottom-left bottom bottom-right
```

The function loops through each board cell and checks every combination of:

```
dr in (-1, 0, 1)
dc in (-1, 0, 1)
```

The pair `(0, 0)` is skipped because it means staying on the same cell. Positions outside the 4x4 board are also skipped.

It returns a map such as:

```
{
    0: [1, 4, 5],
    ...
}
```

Cell 0 is the top-left corner, so it has only three neighbors. A middle cell has up to eight neighbors.

This map is calculated once per algorithm call in the current implementation. It is small because the board has only 16 cells. An optimization could compute it once in `benchmark` and reuse it, but the current version is simpler to explain.

---

11. Paths and coordinates: `path`

```
def path(cells: list[int]) -> list[tuple[int, int]]:
```

Search functions track cells by their flat indexes. This helper converts them to `(row, column)` coordinates.

For example:

```
path([0, 1, 2])
```

returns:

```
[(0, 0), (0, 1), (0, 2)]
```

The UI displays these as one-based coordinates:

```
(1, 1) -> (1, 2) -> (1, 3)
```

One path is stored for each word. If a word can be found in multiple ways, the first path found by that algorithm is kept.

---

12. Trie data structure

What is a trie?

A trie, also called a prefix tree, stores words character by character.

For words:

```
CAT
CATS
CAR
```

the first letters share one `C` node, and the next letters share one `A` node.
The common prefix is stored once.

This is useful because Boggle searches many partial strings. A trie can answer the question:

```
Can this partial string become a dictionary word?
```

by checking whether the corresponding child node exists.

`Trie` fields

```
class Trie:
    def __init__(self):
        self.children = {}
        self.word = None
        self.left = 0
```

`children`

A map from a character to the next trie node.

`word`

Stores the complete word when this node represents the end of a word. It is `None` for non-terminal nodes.

`left`

Counts how many words remain in the subtree rooted at this node. This field is used by the adaptive subtree-pruning algorithm.

`add`

```
def add(self, word: str) -> None:
```

This walks through every character in a word. Missing child nodes are created. The `left` counter is incremented on the root and every node along the path.

`make_trie`

```
def make_trie(words: set[str]) -> Trie:
```

This creates an empty trie and inserts every candidate word.

Trie construction is preprocessing. It is measured separately from search in the benchmark.

---

13. Common search concepts

All search algorithms use depth-first search, or DFS, except that the dictionary-driven algorithm performs a separate DFS for each target word.

Depth-first search

DFS follows one path as far as possible before backtracking.

For example, starting at `C`:

```
C -> A -> T
```

If the path cannot continue, DFS returns to the previous cell and tries another neighbor.

Backtracking

The search must try many possible paths. It does not permanently remove a board cell from the board. Instead, it passes a `visited` representation to the recursive call.

When recursion returns, the previous call can try a different neighbor. This is backtracking.

Bit masks

The code represents visited cells as an integer:

```
1 << cell_index
```

If cell 3 is visited, bit 3 is set.

To test whether a cell is visited:

```
used & (1 << next_cell)
```

To mark it visited:

```
used | (1 << next_cell)
```

This is compact and fast. A 16-cell board needs only 16 bits.

Why cells cannot be reused

Standard Boggle paths cannot use the same cell twice in one word. The bit mask enforces that rule.

---

14. Shared benchmark wrapper: `run`

```
def run(name, board, dictionary, minimum, search) -> Result:
```

This function gives most algorithms a common measurement process.

Preparation phase

```
candidates = {
    w for w in dictionary
    if minimum <= len(w) <= 16
}
```

Only words between the selected minimum length and 16 are kept. A 4x4 board has 16 cells, so a path cannot contain more than 16 letters.

The board is flattened:

```
chars = sum(board, [])
```

The preparation timer stops after this work.

Search phase

The selected search function receives:

- `chars`;
- `candidates`;
- `neighbors()`.

It returns:

```
found_words, states, checks, pruned
```

The wrapper records search time and returns a `Result`.

The frequency-filtered algorithm needs extra preprocessing information, so `benchmark` measures it separately instead of using this wrapper directly.

---

15. Algorithm 1: Brute-force DFS

Function:

```
brute(chars, candidates, adjacent)
```

Main idea

Explore every valid board path up to the longest candidate word. Only after forming a string does the algorithm check whether it is in the dictionary.

Steps

1. Find the maximum candidate word length.
2. Start DFS from every one of the 16 cells.
3. Add the current cell's character to the current string.
4. If the string is a candidate word, save it.
5. Try every unvisited neighboring cell.
6. Stop when the path reaches the maximum candidate length.

What it does not do

It does not check whether a partial string is a valid prefix. For example, if `QZX` cannot begin any dictionary word, brute force may still continue from `QZX` until the maximum length.

That is why it is the baseline and usually explores the most states.

Complexity

For a corner or edge, the number of choices is smaller than eight, but a loose worst-case estimate is exponential:

```
O(16 x 8^L)
```

where `L` is the maximum path length searched. Since the board has only 16 cells, the practical path limit is 16.

The expression is an upper-bound style explanation, not an exact count. The visited-cell rule reduces the number of choices at each level.

Strength

- Simple reference implementation.
- Useful for checking whether optimized algorithms return the same words.

Weakness

- Does not use prefix information.
- Performs a large amount of unnecessary exploration.

---

16. Algorithm 2: Prefix-set DFS

Function:

```
prefix(chars, candidates, adjacent)
```

Main idea

Build a set containing every prefix of every candidate word.

For example, for `CATS`, store:

```
C
CA
CAT
CATS
```

During DFS, extend a path only if the new string is in the prefix set.

Steps

1. Build all prefixes.
2. Start DFS from each board cell.
3. Before recursively continuing, form the next string.
4. Check whether that string is in the prefix set.
5. Prune the branch if it is not a prefix.
6. Record the string if it is a complete candidate word.

Why it is faster

Suppose the current path is `ZX`. If no dictionary word begins with `ZX`, the algorithm stops immediately. There is no reason to try more cells after that.

Complexity

If `P` is the total number of stored prefix strings, preprocessing is roughly related to the total length of the candidate words. Search remains exponential in the worst case, but usually explores far fewer states than brute force.

Membership checks in a Python set are average O(1).

Strength

- Easy to understand.
- Fast prefix membership checks.
- Usually much faster than brute force.

Weakness

- Stores many prefix strings.
- Builds new strings during search.
- A trie can represent shared prefixes more compactly.

---

17. Algorithm 3: Trie DFS

Function:

```
trie_search(chars, candidates, adjacent)
```

When called without `prefixes`, this function builds a trie.

Main idea

Instead of constructing a complete prefix string and looking it up in a set, follow the trie child corresponding to the next board character.

If the child does not exist, the path cannot form a dictionary word and is pruned.

Steps

1. Build a trie from all candidate words.
2. Start from every board cell whose character appears under the trie root.
3. At each DFS step, inspect the current trie node.
4. If the node represents a complete word, record it.
5. For every unvisited neighboring cell:
    - look for a child node matching its character;
    - prune if no child exists;
    - otherwise recurse into that child.

Difference from prefix-set DFS

Both algorithms use prefix pruning. The difference is the prefix data structure:

Prefix-set DFS	**Trie DFS**
Stores complete prefix strings	Stores characters as linked nodes
Checks `next_string in prefixes`	Checks `node.children.get(character)`
Simpler conceptually	Shares common prefixes

Complexity

Trie construction is approximately proportional to the total number of characters inserted, although shared prefixes reduce actual node count.

Search is still exponential in the worst case because the board can have many paths, but invalid prefixes are rejected early.

Strength

- Efficient prefix transitions.
- Shares common prefixes.
- Does not need to build a separate prefix string set.

Weakness

- The trie uses additional object and dictionary memory.

- It is slightly more complex to explain than a set.

---

18. Algorithm 4: Dictionary-driven DFS

Function:

```
dictionary_search(chars, candidates, adjacent)
```

Main idea

Reverse the search direction:

- Instead of generating board paths and asking whether they form words,
- take each dictionary word and ask whether the board can spell it.

Steps

1. Group board cell indexes by their starting character.
2. For each candidate word:
   - find board cells matching its first letter;
   - start a recursive match from each possible starting cell;
   - compare the next required word character with neighboring cells;
   - stop as soon as one complete path is found.
3. Save the first successful path.

Example

For the word `CAT`:

1. find all cells containing `C`;
2. from each `C`, find an adjacent `A`;
3. from that `A`, find an adjacent `T`;
4. stop when `CAT` is matched.

Complexity

If:

- `W` is the number of candidate words;

- `L` is the maximum word length;
- `B` is the branching factor;

then a loose estimate is:

```
O(W x 16 x B^L)
```

It can be competitive when the dictionary is small, but it is often less efficient when the dictionary is large because it starts a separate search for every word.

Strength

- Directly answers whether each dictionary word exists.
- Can stop searching a word after finding its first path.

Weakness

- Repeats similar board work for words with common prefixes.
- Performance depends strongly on dictionary size.

---

19. Algorithm 5: Frequency-filtered Trie DFS

Function:

```
frequency(chars, candidates, adjacent)
```

The `benchmark` function performs the same logic directly so that preparation and search timings can be measured separately.

Main idea

Before searching, remove every word that requires a character more times than the board contains it.

Example:

```
Board has one Q.
Word requires two Q characters.
The word is impossible and can be removed immediately.
```

Steps

1. Count each character on the board.
2. For each candidate word:
   - count its letters;
   - compare each word count with the board count;
   - reject it if any requirement is too large.
3. Build and search a trie containing only the remaining words.

Important limitation

Frequency filtering is necessary but not sufficient.

A word can have valid letter counts but still be impossible because the letters are not adjacent in the required order. For that reason, trie DFS is still needed after filtering.

Complexity

The filtering pass is approximately proportional to the total number of characters in the candidates. The remaining search has trie DFS complexity, usually over a smaller candidate set.

Strength

- Removes obviously impossible words cheaply.
- Reduces trie size and search work.

Weakness

- Counts ignore adjacency and letter order.
- It adds preprocessing time.
- If the dictionary is already tiny, filtering may not be worthwhile.

---

20. Algorithm 6: Trie DFS with subtree pruning

Function:

```
adaptive(chars, candidates, adjacent)
```

Main idea

This algorithm uses a trie plus a dynamic count of words remaining below each trie node.

When a word is found, its count is removed from every node on its trie path. If a node's remaining count reaches zero, the entire subtree can be skipped in future searches.

Example

Suppose a trie branch represents:

```
CAT
CATS
CATER
```

After all three words have been found, there is no need to search that branch again. Its `left` value becomes zero.

Steps

1. Build a trie and initialize `left` counts.
2. Start DFS from cells whose letters exist under the root.
3. When a terminal trie node is reached:
   - save the word and path;
   - call `remove(word)`;
   - set the terminal node's `word` to `None`.
4. Before exploring a node's children, stop if `node.left` is zero.
5. Before entering a child, stop if the child is missing or exhausted.

`remove`

`remove(word)` walks from the root through the word's characters and decrements `left` on every node. This updates the number of undiscovered words in every affected subtree.

Complexity

The worst case is still exponential, but the practical search can be smaller because completed subtrees disappear dynamically.

Strength

- Combines trie prefix pruning with result-aware pruning.
- Can avoid repeating work after all words in a branch are found.

Weakness

- More complicated than ordinary trie DFS.
- Mutates the trie during searching.
- The algorithm must carefully decrement exactly the paths of discovered words.

---

21. How `benchmark` runs the six algorithms

```
def benchmark(board, dictionary, minimum):
```

It creates a list of six algorithm names and functions:

```
[
    ("Brute-force DFS", brute),
    ("Prefix-set DFS", prefix),
    ("Trie DFS", ...),
    ("Dictionary-driven DFS", dictionary_search),
    ("Frequency-filtered trie DFS", frequency),
    ("Trie DFS with subtree pruning", adaptive),
]
```

The algorithms are then run one after another.

For most algorithms, `run` handles:

- candidate filtering;
- board flattening;
- timing;
- invoking the algorithm;
- creating the result object.

The frequency-filtered version has its own block because it needs to report both:

- time spent counting and filtering letters;
- time spent doing the trie search;
- number of words filtered.

The final return value is a list of six `Result` objects.

---

22. Streamlit interface flow

The interface is created by:

```
def app():
```

Title and description

`st.title` and `st.write` introduce the project.

Board input

The board is built inside a form:

```
with st.form("benchmark"):
```

For each of the four rows:

1. `st.columns(4)` creates four visual columns;
2. one `st.text_input` is placed in each column;
3. `max_chars=1` limits each cell to one character;
4. a stable key such as `cell-0-2` preserves the cell's value across reruns.

The sixteen values are joined into four strings:

```
board_text = "\n".join(...)
```

Minimum length

The slider chooses a value from 3 to 16. Words shorter than this are excluded.

Dictionary upload

`st.file_uploader` optionally provides a text file. If no file is uploaded, the bundled dictionary is read.

Submit button

The algorithms run only when the user clicks `Run benchmark`.

---

23. Session state

Streamlit reruns the Python script when a widget changes. Without session state, the results would disappear after selecting a word or interacting with the page.

After a successful benchmark, the program stores:

```
st.session_state.results
st.session_state.board
st.session_state.dictionary_size
st.session_state.rejected
```

On later reruns, the program checks:

```
if "results" not in st.session_state:
    return
```

If results exist, the table, charts, and word-path viewer are rendered again.

This separates:

- running the expensive benchmark;
- displaying and exploring its results.

---

24. Results table

Each `Result` is converted into a dictionary:

```
row = asdict(result)
```

The UI then adds friendly table fields:

- `algorithm`
- `preprocessing_ms`
- `search_ms`
- `words_found`

The internal `words` dictionary is removed from the table because paths are displayed separately.

The table lets us compare:

- preparation time;
- search time;
- DFS states;
- neighbor checks;
- branches pruned;
- dictionary words filtered;
- words found.

---

25. Performance charts

Pandas creates a small chart data frame with:

- algorithm name;
- total time;
- DFS states;

- branches pruned.

The total time is:

```
preparation time + search time
```

The three bar charts show:

1. total benchmark time;
2. DFS states explored;
3. branches pruned.

How to interpret the charts

- Fewer states generally means less search work.
- More pruning is not automatically better; an algorithm may spend extra preprocessing time to achieve it.
- The fastest algorithm depends on the board, dictionary, and minimum length.
- A timing result from one run is not a universal ranking. Repeat runs can vary because of the operating system and Python runtime.

For a fair experiment, keep the following constant when comparing algorithms:

- board;
- dictionary;
- minimum word length;
- Python environment;
- machine conditions.

---

26. Word-path viewer

The viewer uses the first algorithm's word map:

```
results[0].words
```

All algorithms should find the same set of words. The first result is used as the display source because it is simple and gives one valid path for each word.

The user selects a word. The program:

1. gets its list of coordinates;
2. converts the list to a set for fast membership checking;
3. loops through all 16 board cells;
4. uses `st.success` for cells in the path;
5. uses `st.info` for other cells;
6. prints the ordered coordinates below the grid.

The set is only used for highlighting. The original list is used for the ordered coordinate text.

---

27. Correctness and agreement between algorithms

The algorithms use different search strategies, but they should produce the same set of valid words.

The reason they can agree is that all of them enforce the same rules:

1. a word must be in the candidate dictionary;
2. its length must meet the minimum;
3. it cannot be longer than 16;
4. adjacent cells must be used;
5. a cell cannot be used twice in one path.

The optimized algorithms only remove paths that are provably impossible:

- missing prefixes;
- missing trie transitions;
- impossible character counts;
- exhausted trie subtrees.

They do not remove valid paths.

The brute-force algorithm is a useful reference implementation. If an optimized algorithm returns a different word set, that is an indication of a bug in the optimization or its pruning condition.

28. Why the default board finds the same words

The default board is:

```
C A T S
R A T E
L I O N
P E N D
```

With the bundled dictionary and minimum length 3, the current implementation finds 51 words with each algorithm.

The exact timing and search counters can change slightly between runs, but the word sets should agree.

---

29. Complexity summary

Let:

- `N = 16`, the number of board cells;
- `L`, the maximum searchable word length;
- `W`, the number of candidate dictionary words;
- `P`, the number of distinct prefixes;
- `B`, the average number of available neighboring cells.

Algorithm	Main work	Practical improvement
Brute-force DFS	Explore possible paths	No prefix pruning
Prefix-set DFS	DFS plus prefix set lookup	Stop at invalid prefixes
Trie DFS	DFS plus trie transitions	Shared prefix representation
Dictionary-driven DFS	Search separately for each word	Stops after a word is matched
Frequency-filtered Trie DFS	Count letters, then Trie DFS	Remove impossible letter counts

Subtree-pruned Trie DFS	Trie DFS plus remaining-word counts	Remove completed subtrees

Loose worst-case forms:

```
Brute-force DFS:              O(N x B^L)
Prefix-set DFS:               O(N x B^L) worst case, less in practice
Trie DFS:                     O(N x B^L) worst case, less in practice
Dictionary-driven DFS:       O(W x N x B^L)
Frequency filtering:         O(total candidate letters) + Trie DFS
Subtree-pruned Trie DFS:     Exponential worst case, lower practical
work
```

The important point is not that the optimized algorithms change the theoretical worst case completely. Their main benefit is reducing the number of paths actually explored on realistic dictionaries and boards.

---

30. Space complexity

Board and adjacency

The board has a fixed 16 cells. The adjacency map is also constant-sized for this project.

Dictionary

The dictionary set uses O(W) space.

Prefix set

The prefix-set algorithm uses O(P) additional space.

Trie

The trie uses one node for each distinct prefix character position. Its space is proportional to the total characters inserted, reduced by shared prefixes.

DFS recursion

The recursion depth is at most 16 because a board cell cannot be reused.

Paths

The results store one path per discovered word, so path storage is related to the number of found words and their lengths.

---

31. Important implementation assumptions

The project intentionally uses these fixed assumptions:

- board size is exactly 4x4;
- movement includes all eight surrounding directions;
- a cell cannot be reused within one word;
- dictionary words contain ASCII letters only;
- words are compared in uppercase;
- the maximum possible word length is 16;
- the minimum word length is selected between 3 and 16;
- one valid path is stored for each word, not every possible path.

If the project is changed to support another board size, code using the fixed values `4`, `16`, or `range(16)` must also be updated.

---

32. Why only one path is shown

A word may be formable through multiple different paths. Storing every path could make the output very large and would not be needed for the benchmark's main purpose.

The program uses:

```
found.setdefault(word, path(cells))
```

`setdefault` stores the first path found and leaves it unchanged if another path for the same word is found later.

This is enough to prove that the word is formable and to display one path.

---

33. Common viva questions and answers

Q1. What problem does the project solve?

It finds all dictionary words that can be formed on a 4x4 Boggle board using adjacent cells, including diagonals, without reusing a cell in the same word. It also compares six search strategies for the same problem.

Q2. Why is DFS suitable here?

Every possible word is a path through the board. DFS naturally explores one path at a time and backtracks when a path ends or becomes invalid.

Q3. What is backtracking?

Backtracking means choosing a possible move, recursively exploring it, then undoing that choice logically by returning to the previous recursive call and trying another move.

Q4. Why is the board represented by a bit mask?

A 4x4 board has only 16 cells. One bit can represent each cell's visited state. Bit operations are compact and fast compared with copying a full visited matrix at every recursive call.

Q5. Why can a word not reuse a cell?

That is a rule of Boggle. The visited bit mask prevents a recursive path from entering a cell whose bit is already set.

Q6. Why are diagonal moves allowed?

Boggle adjacency includes the eight surrounding positions, not only the four horizontal and vertical positions.

Q7. What is the purpose of the dictionary?

The board can generate many letter sequences. The dictionary tells the program which sequences count as valid words.

Q8. Why is the dictionary stored in a set?

Sets remove duplicates and provide average O(1) membership checks.

Q9. What is the difference between a word and a prefix?

A word is a complete dictionary entry. A prefix is an initial part of one or more words. For example, `CAT` is a word, and `CA` is a prefix of `CAT`.

Q10. Why does prefix pruning improve DFS?

If a partial path is not a prefix of any dictionary word, no extension of that path can become a valid word. The whole branch can be stopped immediately.

Q11. What is a trie?

A trie is a tree that stores words character by character. Common prefixes share nodes, making prefix transitions efficient.

Q12. Why can Trie DFS be faster than prefix-set DFS?

Both prune invalid prefixes, but Trie DFS follows an existing character edge instead of constructing a complete prefix string and looking it up in a set. The trie also shares common prefixes.

Q13. Why is brute force included if it is slow?

It is a baseline. Because it explores broadly and does little pruning, its results can be used as a correctness reference for the optimized algorithms.

Q14. What does dictionary-driven search do differently?

It takes one dictionary word at a time and searches the board for that exact letter sequence, rather than generating board paths first.

Q15. Why might dictionary-driven search be slow?

It repeats board searches for many dictionary words. Words with shared prefixes may cause similar work to be repeated.

Q16. What does frequency filtering prove?

It proves that a word is impossible when it needs a letter more times than the board contains that letter.

Q17. Is frequency filtering enough to prove a word is possible?

No. Letter counts do not prove that the letters are adjacent in the required order. The word must still pass the board search.

Q18. What is subtree pruning?

When all words below a trie node have already been found, the node is marked as exhausted. Future searches skip that whole subtree.

Q19. What does `left` mean in the trie?

It is the number of undiscovered words in the subtree rooted at that node. It allows the adaptive algorithm to know whether a branch still has useful work.

Q20. Why is `node.word` set to `None` after discovery?

This prevents the same terminal word from being recorded repeatedly and works with the `left` counters to mark completed trie branches.

Q21. Why are preprocessing and search times separate?

Some algorithms spend time building prefix sets, tries, or frequency counts. Separating the phases shows where the time is spent and makes comparisons more meaningful.

Q22. Is the fastest algorithm always the best algorithm?

No. Runtime depends on the board, dictionary, minimum length, machine, and implementation. A good algorithm should also be correct, understandable, and appropriate for the input size.

Q23. Why can timing change between two runs?

The operating system, Python interpreter, Streamlit, background processes, and cache effects can change the measured time. Small timings are especially noisy.

Q24. How does the program check correctness?

All algorithms receive the same board, dictionary, and word-length rule. Their word sets should match. The brute-force implementation serves as a simple reference for comparing optimized results.

Q25. Why does the app use Streamlit?

Streamlit allows a Python program to create an interactive web interface without writing separate HTML, CSS, or JavaScript.

Q26. Why is session state needed?

Streamlit reruns the script when widgets change. Session state preserves the last benchmark results so selecting a word or viewing the page does not discard the computation.

Q27. Why is only one path stored for a word?

The benchmark needs to prove that the word exists, and the UI needs one path to highlight. Storing every path would use more memory and create unnecessary output.

Q28. What happens if the uploaded dictionary contains duplicates?

Duplicates collapse into one entry because the dictionary is normalized into a set.

Q29. What happens if a board cell is empty?

`board_from` rejects the board because each row must contain exactly four alphabetic characters. The UI displays the validation error.

Q30. Why is the maximum word length 16?

There are only 16 cells and a cell cannot be reused, so a valid path cannot contain more than 16 letters.

---

34. A short presentation explanation

If only a one-minute explanation is needed:

> This project compares six ways to solve a 4x4 Boggle board. The board is represented as 16 cells, and each algorithm uses DFS to explore adjacent cells without reusing a cell. The brute-force version explores paths without prefix knowledge, so it acts as a baseline. Prefix-set and trie versions prune paths that cannot become dictionary words. Dictionary-driven search checks each candidate word directly. Frequency filtering removes words that require unavailable letters, and subtree pruning removes trie branches after all their words are found. The app measures preprocessing time, search time, states, neighbor checks, and pruned branches, then displays the results in Streamlit. All algorithms should return the same word set because they use the same board rules and dictionary; they only differ in how efficiently they search.

---

35. What to emphasize to the professor

The strongest points of the project are:

1. It compares a simple baseline with progressively more optimized methods.
2. It uses the same input and validation rules for every algorithm.
3. It measures both runtime and internal search behavior.
4. It demonstrates DFS, backtracking, hashing, tries, pruning, and bit masks.
5. It checks practical performance rather than discussing complexity only in theory.
6. It keeps the interface in Python through Streamlit.

The most important caveat to state honestly is:

> The complexity of all board-path searches is exponential in the worst case. The optimizations mainly reduce the practical search space by rejecting impossible paths earlier.